Leetcode 214 Shortest Palindrome

```python
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        def isPalindrome(st):
            return st==st[::-1]

        n= len(s)

        for i in range(n, -1 , -1):
            if isPalindrome(s[:i]):
                remaining= s[i:]
                return remaining[::-1] + s
        return s
```

Brute Force

s = "abcd"
n = 4

i = n -> 0

Iteration - 1

$i = 4$

$s[:4] = "abcd"$

Check Pallindrome

abcd != dcba

Not a Pallindrome

Iteration - 2

$i = 3$

$s[:3] = "abc"$

Check

"abc" != "cba"

Not Pallindrome

Iteration 3

$i = 2$

$s[:2] = "ab"$

Check

"ab" != "ba"

Not Pallindrome

Iteration 4

$i = 1$

$s[:1] = "a"$

Check, "a" == "a"

Palindrome found

Remaining Part

remaining = $s[1:]$ = "bcd"

Reverse it

"dcb"

Add in front

Result = "dcb" + "abcd"

= "dcbabcd"

= "dcbabcd"

Time Complexity → $O(n^2)$

Space complexity → $O(n)$

Optimal

```python
class Solution:
    def shortestPalindrome(self, s: str) -> str:
        rev= s[::-1]
        temp=s+"#"+rev
        lps= [0] * len(temp)
        j=0
        for i in range(1, len(temp)):
            while j >0 and temp[i]!= temp[j]:
                j= lps[j-1]

            if  temp[i] == temp[j]:
                j+=1
            lps[i] =j
        longest_prefix= lps[-1]
        return rev[:len(s) - longest_prefix] +s
```

s = "abcd"

rev = "dcba"

temp = s + "#" + rev

= "abcd#dcba"

Step 2 Build LPS Array

temp = abcd # dcba

index = 0 1 2 3 4 5 6 7 8

lps = [0, 0, 0, 0, 0, 0, 0, 0, 0]

j = 0

i = 1

temp[1] = b

temp[j] = a

Not equal

lps[1] = 0

i = 2

temp[2] = c

temp[0] = a

Not equal

lps[2] = 0

i = 3

d vs a

Not Equal

lps[3] = 0

i = 4

vs a

Not equal

lps [4] = 0

i = 5

d vs a

Not equal, lps[5] = 0

i = 6

c vs a

Not Equal

lps [6] = 0

i = 7

b vs a

Not Equal

lps [7] = 0

i = 8

a vs a

Match

$j = 1$

$lps[8] = 1$

Final LPS

$lps = [0, 0, 0, 0, 0, 0, 0, 0, 1]$

Last value

longest_prefix = 1

Meaning

Longest Pallindrome Prefix Length

Step 3 : Build Answer

Characters to add

↳ len(s) − longest_Prefix

$= 4 - 1$

$= 3$

rev[:3] = "dcb"

Add in front

"dcb" + "abcd"

"dcbabcd"

Time complexity :- O(n)

Space complexity :- O(n)